Slither Report: Polydash Launchpad

```
slither .        # uses slither.config.json
```

- Slither 0.11.6, 100 detectors, run on `src/` only. `slither.config.json` filters out `lib/`, `test/` and `script/`, and excludes the informational detectors `timestamp` and `low-level-calls`.
- **Before: 50 findings. After: 0 findings** (“62 contracts with 100 detectors, 0 result(s) found”).
- If you run it without the exclusions, about 13 `timestamp` and 7 `low-level-calls` informational findings remain. Both are intentional: the timestamps drive the timelocks, and the low-level calls are ETH transfers whose success flag is checked.
- This is a static-analysis triage, **not an audit**. An external audit is still required before mainnet.

Real bugs found during this pass (fixed, with regression tests)

#	Severity	Bug	Fix	Test
1	**High**	**Stranded commitments.** Suppose a launch had more than 200 committers and graduation happened inside the first `goLive` batch. The later commitments could then never be processed or withdrawn, because `processCommitmentBatch` reverted with `WrongPhase(Graduated)`.	`processCommitmentBatch` now keeps working in the Graduated phase while `commitmentCursor < committers.length`, and refunds the rest. It also adds `withdrawUnprocessedCommitment()`, which lets a committer pull their own ETH after graduation.	`test_GraduationMidBatchRefundsRemainingCommitters`, `test_WithdrawUnprocessedCommitmentAfterGraduation`, `test_WithdrawUnprocessedOnlyAfterGraduation`
2	Low	**The `PolydashLaunch` implementation could be initialized** by anyone. It held no funds, but this is bad hygiene.	The constructor sets `factory = address(this)`, and `initialize` reverts if `factory != address(0)`.	`test_ImplementationCannotBeInitialized`
3	Medium (design)	**Graduation ran inside `_executeBuy`.** That meant external calls happened in the middle of a buy or commitment loop, and the code did not follow checks-effects-interactions (CEI).	Graduation now runs once, at the **end** of `buy`, `goLive` and `processCommitmentBatch`, via `_thresholdReached()`. `_graduate` writes all state and emits its event **before** the external calls (burn, ETH transfer, `deployGraduatedPool`). It	Existing graduation tests, plus a fork test against the real V4 PoolManager

#	Severity	Bug	Fix	Test
			takes the pool id from the new `hook.computePoolId(token)` and then verifies the returned id and that liquidity is greater than 0. If the threshold is reached during commitment processing, the Trading phase is never opened.	

The hook's `_charge` now emits its event before `take` / `donate`, which is CEI inside the PoolManager lock.

Suppressed findings (inline `// slither-disable-next-line`, each with a reason)

Detector	Where	Why it is safe
arbitrary-send-eth	`Factory.collectPlatformFees`, `Launch._graduate`	The destinations are the owner-set fee wallet (48h timelock), the buyback router, and the hook fixed at `initialize`. Callers cannot choose any of them.
reentrancy-eth / -no-eth / -benign / -events	Hook ↔ Launch graduation, `executePayoutWalletChange`, hook `_charge`	Every entry point shares one `ReentrancyGuard`. The callees are the trusted hook, the plain OZ ERC-20 token, or the PoolManager, which runs inside its own lock. The payout-wallet change claims old fees to the **old** wallet on purpose. Covered by `Reentrancy.t.sol` and `CommitVault.t.sol` (malicious receivers).
unused-return	V4 `initialize`, `settle`, `donate`, `modifyLiquidity`, `getPositionInfo`, `observe`, buyback	The return values are not needed: we pay exact deltas, and the amounts are emitted elsewhere.
incorrect-equality	`buy` / `sell` `== 0` guards	Slippage guards on values computed in the same call. An attacker cannot force them.
write-after-write	Hook `_deploying` flag	A scoped flag that is read inside `unlockCallback`. Slither does not see that read.
calls-loop	`collectPlatformFeesMany`, vault `settle`	The caller supplies the batch and pays the gas. If a buyback fails, the funds fall back to the fee wallet.
missing-zero-check	`setTradingRewardsPool(0)`	By design: passing zero disables the buyback.

The new commit contract (`PolydashCommitVault`)

Slither reports 0 findings on it. It has no owner, no fee, no pause and no sweep. Its `receive()` only accepts ETH from `factory.isLaunch` addresses. Payouts are pro-rata, and the last claimer receives the exact remainder, so no dust is left behind. It has 21 unit, fuzz and reentrancy tests, and it is also exercised end to end in the Robinhood mainnet fork test.

Verification

Check	Result
`slither .`	0 findings
`forge test`	**136 / 136 passing** (12 suites)
`FORK_TESTS=true forge test --match-path "test/fork/*"`	Pass. Robinhood mainnet fork, real V4 Pool-Manager and CREATE2 deployer, full flow from vault to graduation to V4 swaps to fees
`forge coverage` (src)	Lines 98.66%, statements 95.06%, branches 75.58%, functions 98.97%